

Αλγόριθμοι και Πολυπλοκότητα

ΦΡΟΝΤΙΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ (10)

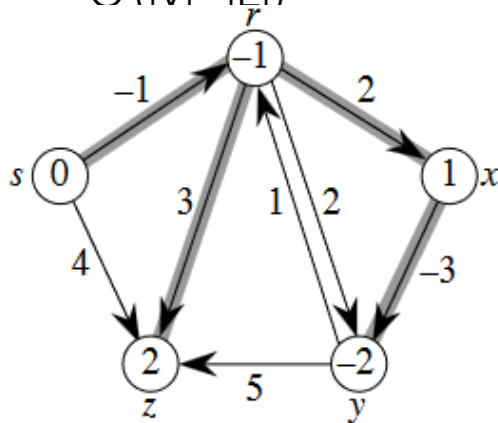
Δυναμικός προγραμματισμός (2/2)

4/6/2024

Έφη Μαλέσιου
Διδακτορική φοιτήτρια

Bellman-Ford

- συντομότερα μονοπάτια
- εφαρμόζεται και με αρνητικά βάρη, σε κατευθυνόμενο γράφο
- επιστρέφει: True, όταν δεν υπάρχει κύκλος αρνητικού συνολικού κόστους από την κορυφή s ή
False, όταν υπάρχει
 - ακμή (u,v) : αν $d[v] > d[u] + c(u,v)$, υπάρχει (μετά το τέλος των «κανονικών επαναλήψεων» - μείωση κόστους σε μία επιπλέον επανάληψη)
- $O(|V| \cdot |E|)$

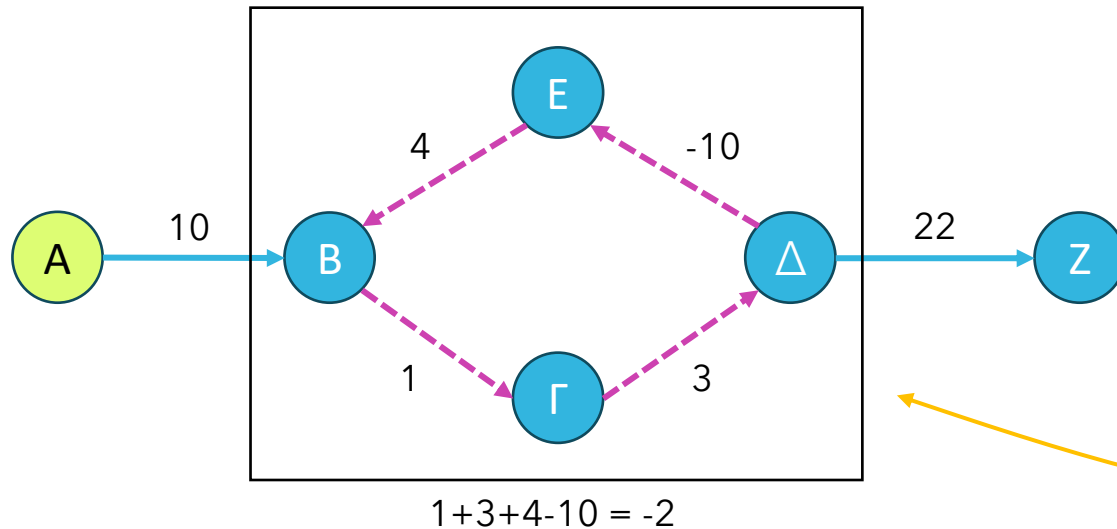


- $|V|$ κορυφές, έως $|V|-1$ επαναλήψεις, όλες οι ακμές
- +1 επανάληψη για τον εντοπισμό αρνητικού κύκλου που φτάνουμε από την κορυφή έναρξης s
- αρχικό κόστος: $s = 0$, κάθε άλλη $= \infty$
- ανά επανάληψη:
 - εξέταση όλων των ακμών
 - σε κάθε κόμβο, ποιες είναι οι εξερχόμενες ακμές
 - αν δεν αλλάξει καμία τιμή, σταματάμε

```
BELLMAN-FORD( $V, E, w, s$ )
INIT-SINGLE-SOURCE( $V, s$ )
for  $i \leftarrow 1$  to  $|V| - 1$ 
    do for each edge  $(u, v) \in E$ 
        do RELAX( $u, v, w$ )
for each edge  $(u, v) \in E$ 
    do if  $d[v] > d[u] + w(u, v)$ 
        then return FALSE
return TRUE
```

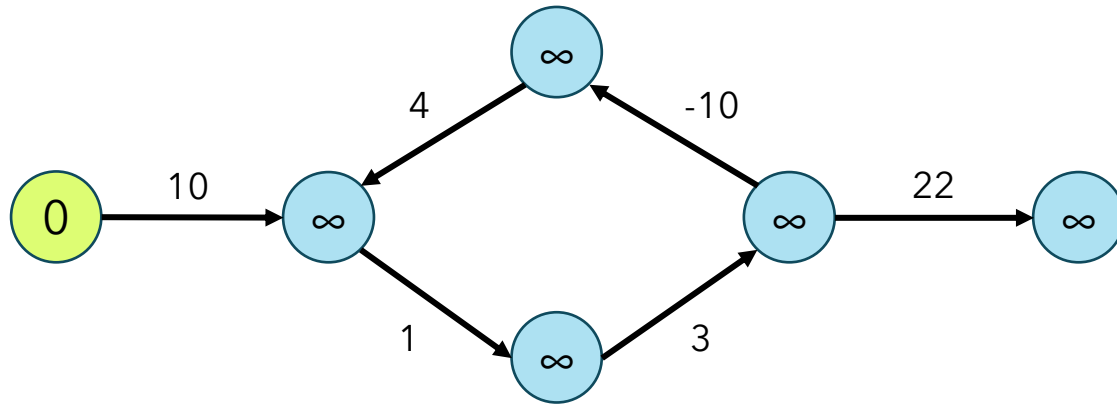
Bellman-Ford

- 1^η επανάληψη: συντομότερα μονοπάτια με 1 ακμή
- i-οστή επανάληψη: συντομότερα μονοπάτια με i ακμές το πολύ
- Σε κάθε επανάληψη ελέγχουμε όλες τις ακμές
- Στο τέλος των $|V-1|$ επαναλήψεων, αν το κόστος μπορεί να μειωθεί περισσότερο, υπάρχει κύκλος με αρνητικό άθροισμα βαρών
 - αν δεν υπάρχει αλλαγή σε κάποιο κόστος μεταξύ 2 διαδοχικών επαναλήψεων, η διαδικασία μπορεί να σταματήσει

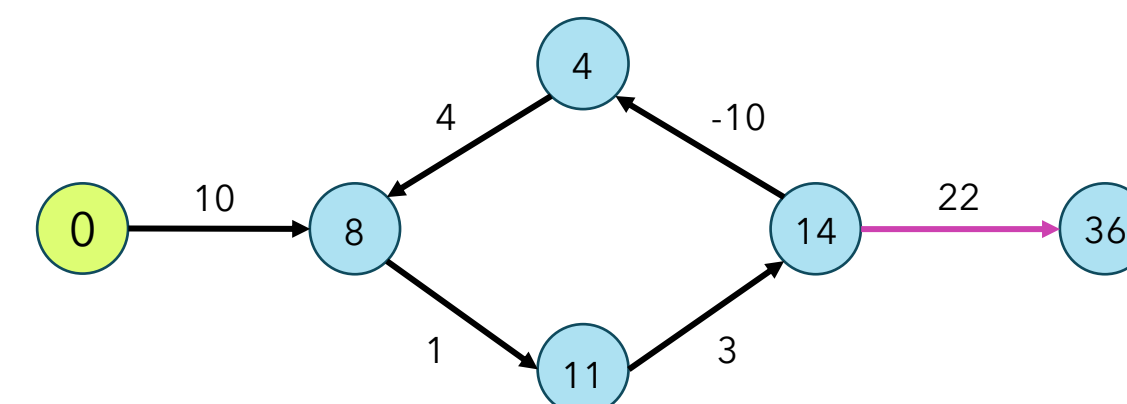
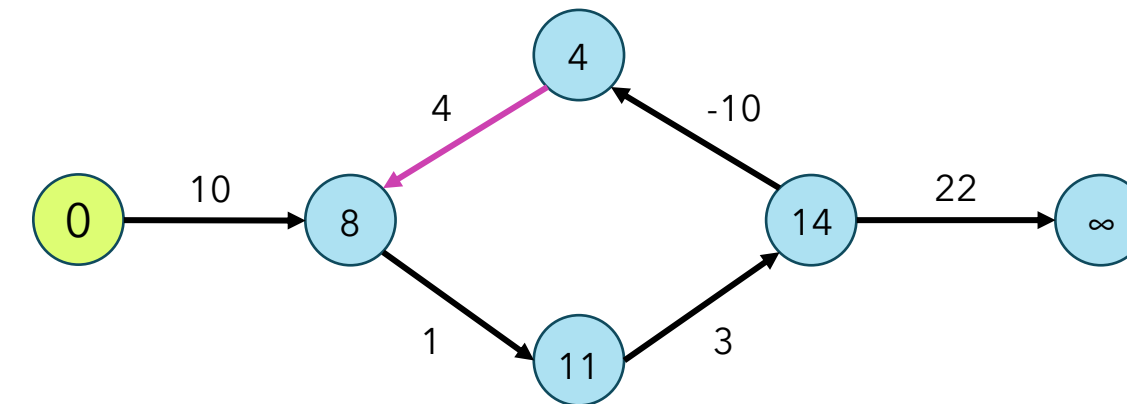
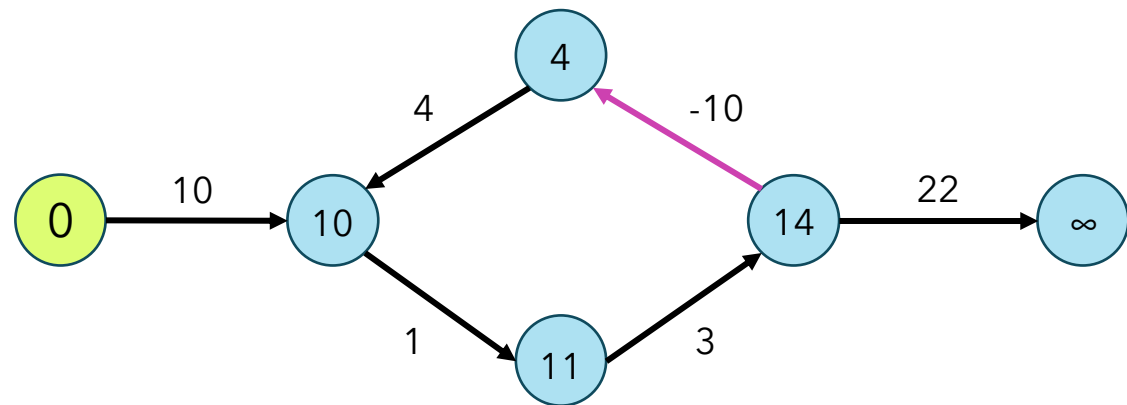
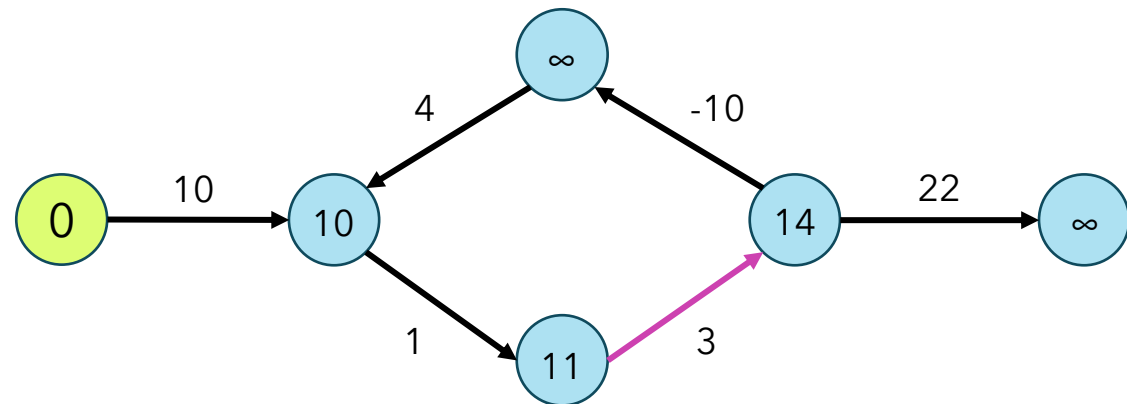
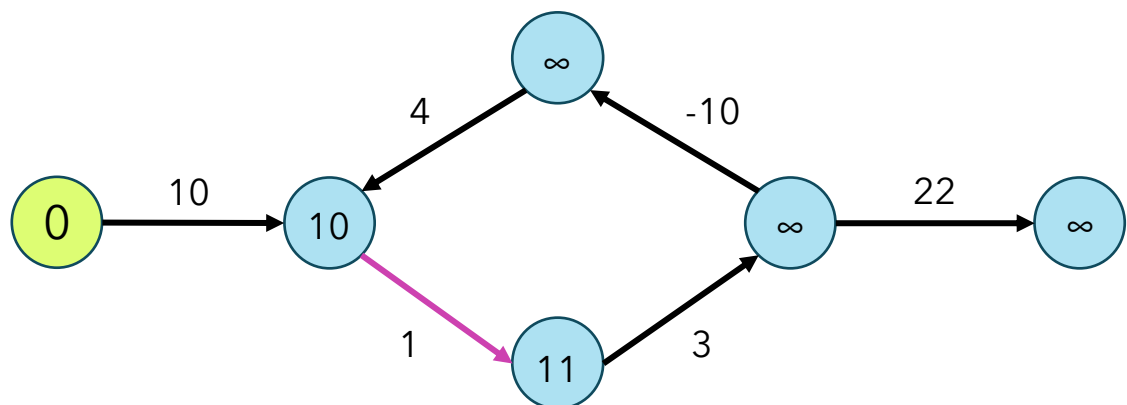
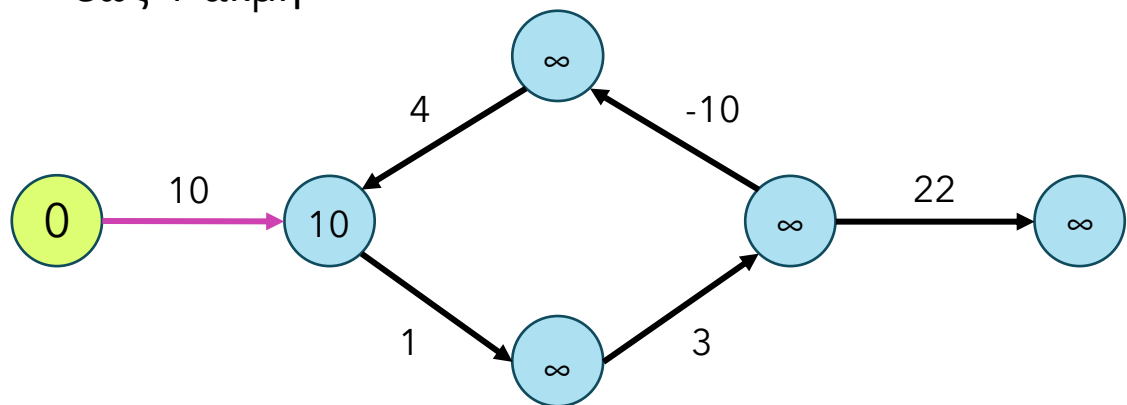


ΕΠΑΝ.	A	B	Γ	Δ	Ε	Ζ
0	0	∞	∞	∞	∞	∞
1	0	8	11	14	4	36
2	0	6	9	12	2	34
3	0	4	7	10	0	32
4	0	2	5	8	-2	30
5	0	0	3	6	-4	28
5+1	0	0	1			

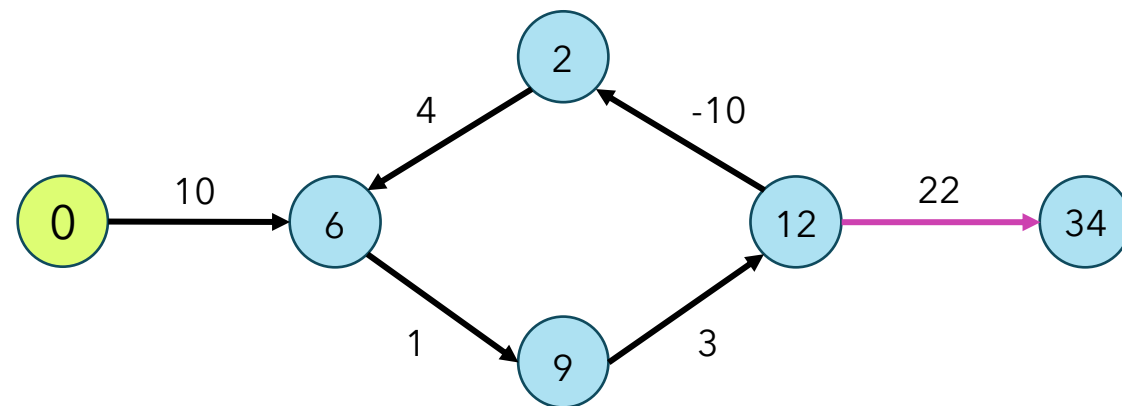
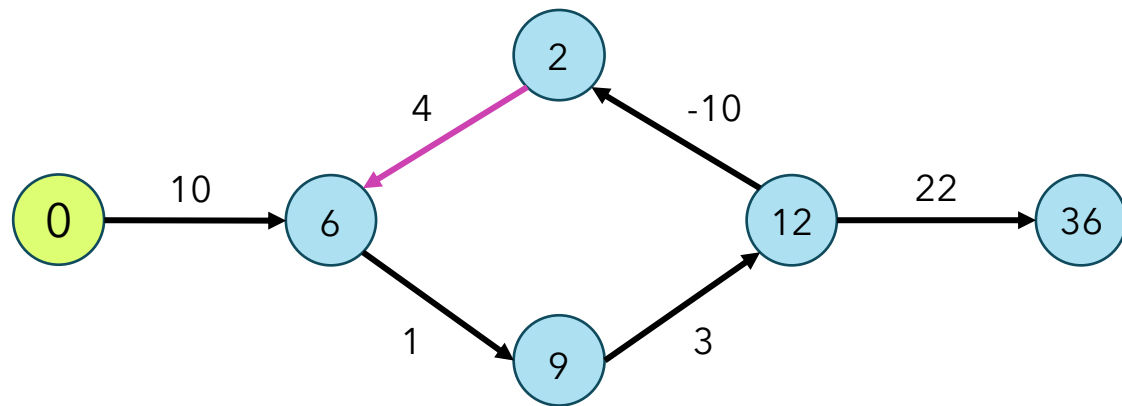
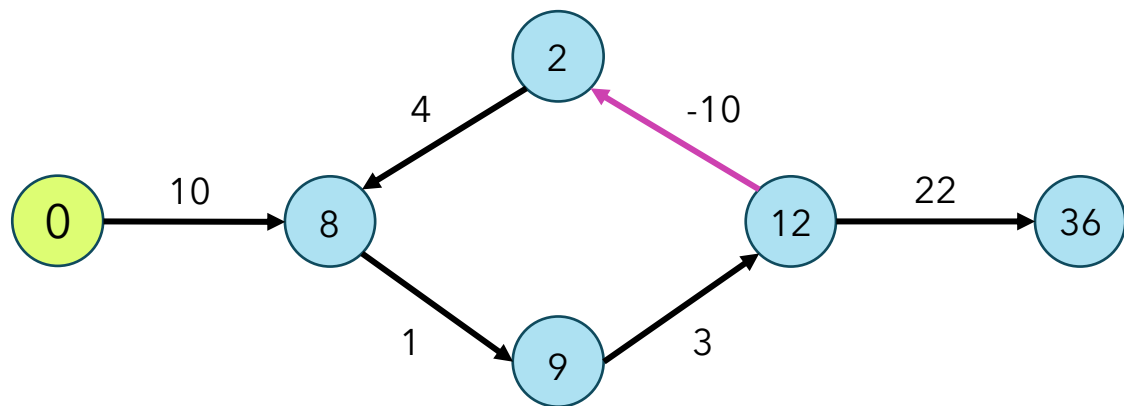
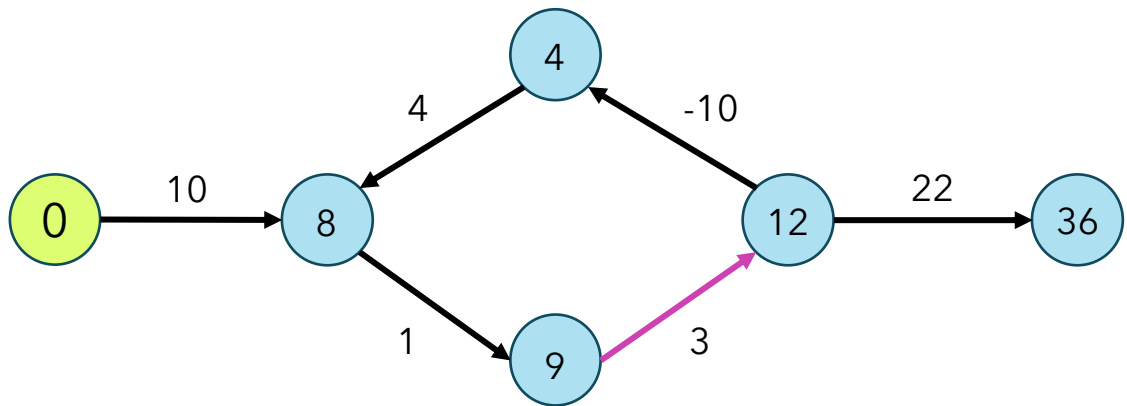
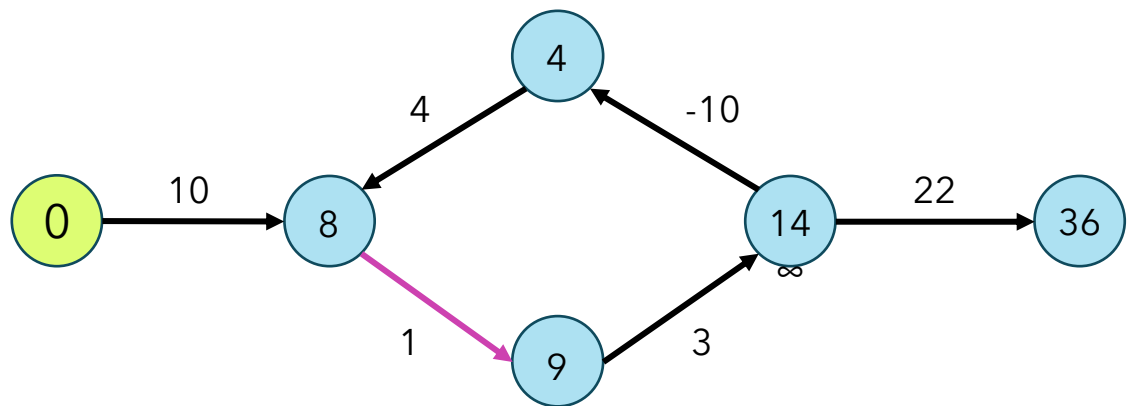
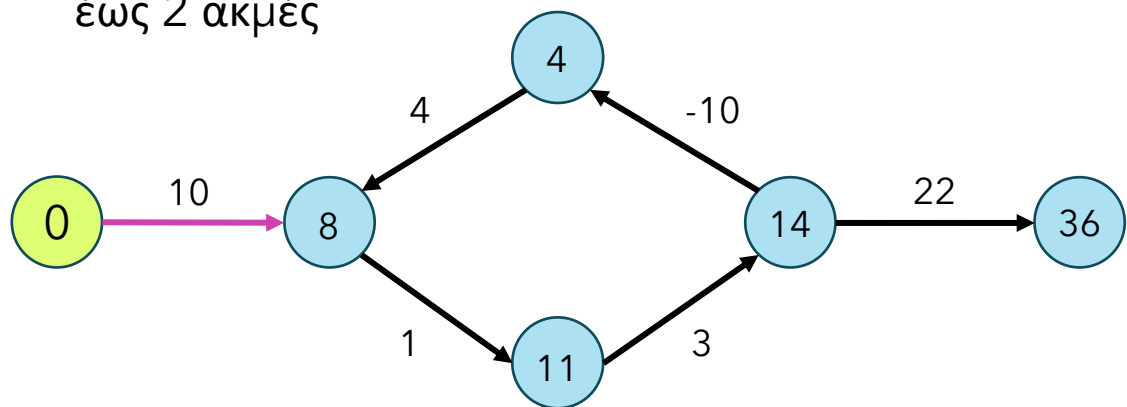
0 ακμές



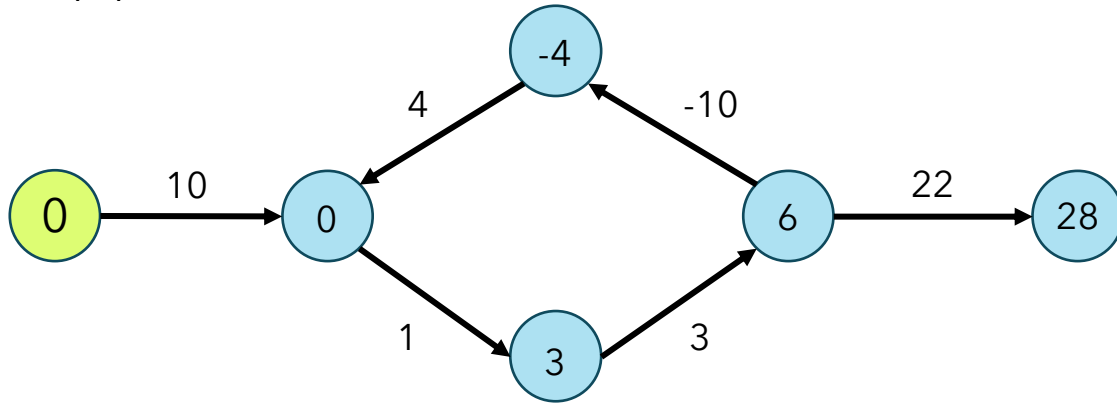
έως 1 ακμή



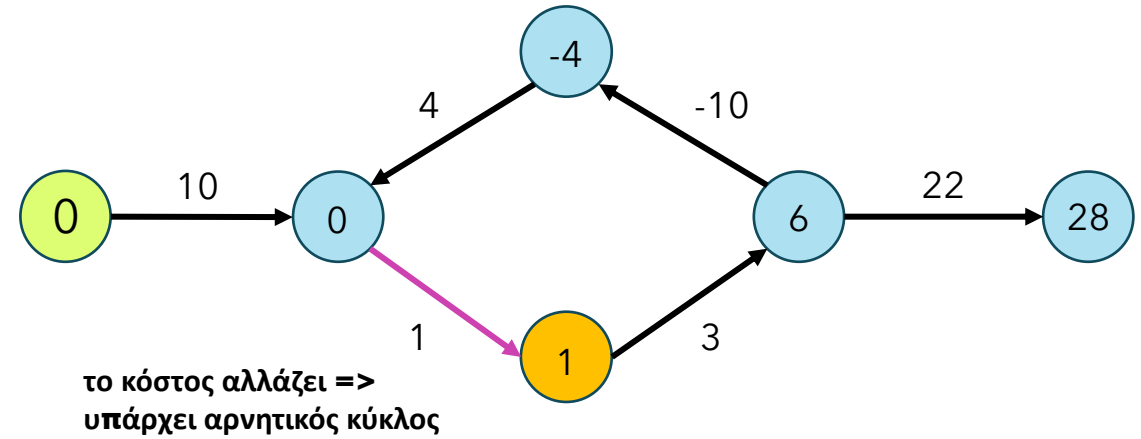
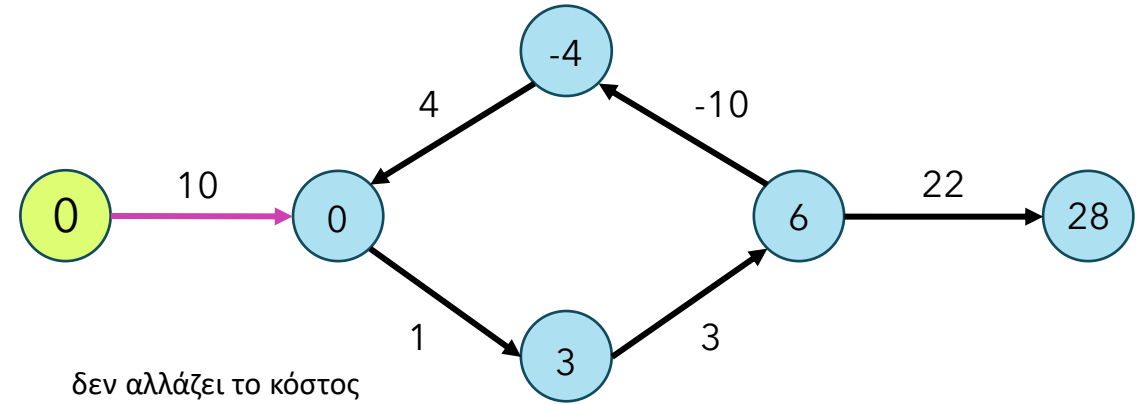
έως 2 ακμές



...έως 5 ακμές
 $|V|-1$ επαναλήψεις



+1 επανάληψη
ανίχνευση αρνητικού κύκλου





Θέμα 3 [35 μονάδες]

Στην κατασκήνωση AlgoRythm αναπτύχθηκε ένα ιδιαίτερο σύστημα ανταλλαγών που περιλαμβάνει το νόμισμα του ευρώ, κάρτες γραφικών, επεξεργαστές, περιφερειακά... n συνολικά είδη (μαζί με το ευρώ). Οι κατασκηνωτές έχουν καταρτίσει έναν κατά το δυνατό δίκαιο πίνακα $R \in A^{n \times n}$, $A = \mathbb{R}^+ \setminus \{0\}$, με τον λόγο ανταλλαγής μεταξύ των ειδών c_i , έτσι ώστε 1 μονάδα του είδους c_i να αγοράζει $R[i, j]$ μονάδες του είδους c_j .

Ο Sheldon πιστεύει πως αυτό ενδέχεται να του αποφέρει κάποιο κέρδος, επειδή παρατήρησε τυχαία κάποια στιγμή ότι με 100 ευρώ μπορούσε να αγοράσει 1 μηχανικό πληκτρολόγιο A , να το ανταλλάξει με 3 ποντίκια B και στη συνέχεια να τα πουλήσει προς 34 ευρώ το ένα, καταλήγοντας να έχει 102 ευρώ και άρα 2% κέρδος! Όπως θα είχε κέρδος αν κατέληγε με περισσότερες μονάδες οποιουδήποτε είδους σε σχέση με αυτές που ξεκίνησε.

Έτσι, θέλει να αναπτύξει έναν αλγόριθμο που αρχικά θα τον πληροφορεί αν αυτό είναι δυνατό και στη συνέχεια να του δίνει αναλυτικά έναν τρόπο με τον οποίο μπορεί να το πετύχει.

1. Προσδιορίστε έναν αποδοτικό αλγόριθμο που να απαντά στο ερώτημα, αν υπάρχει ακολουθία ειδών, $\langle c_{i_1}, c_{i_2}, \dots, c_{i_k} \rangle$, τέτοια ώστε $R[i_1, i_2] \cdot R[i_2, i_3] \cdot \dots \cdot R[i_k, i_1] > 1$. [23 μονάδες]



1. (23 μονάδες) Δημιουργείται ο κατευθυνόμενος γράφος με βάρη, $G = (V, E)$, ως εξής:

- κάθε είδος c_i , παριστάνεται με μία κορυφή $v_i, i = \{1, 2, \dots, n\}$ και προστίθεται ακόμη μία κορυφή έναρξης, v_0
- μεταξύ κάθε ζεύγους διαφορετικών ειδών c_i, c_j , προστίθενται οι δύο κατευθυνόμενες ακμές $(v_i, v_j), (v_j, v_i)$
- επιπλέον, προστίθεται μία κατευθυνόμενη ακμή (v_0, v_i) από την κορυφή έναρξης προς κάθε είδος - κορυφή v_i , με βάρος 0
- το βάρος κάθε άλλης ακμής (v_i, v_j) είναι $\log\left(\frac{1}{R[i,j]}\right) = -\log(R[i,j])$

Συνεπώς, έχουμε $|E| = (n+1) = \Theta(n)$ κορυφές και $|V| = \binom{n}{2} + n = \left(\frac{n!}{2!(n-2)!} + n\right) = \Theta(n^2)$ ακμές.

Ως προς τα βάρη των ακμών, παρατηρείται ότι η επιθυμητή ιδιότητα:

$R[i_1, i_2] \cdot R[i_2, i_3] \dots R[i_{k-1}, i_k] \cdot R[i_k, i_1] > 1$, μπορεί να ισχύει αν και μόνο αν

$\frac{1}{R[i_1, i_2]} \cdot \frac{1}{R[i_2, i_3]} \dots \frac{1}{R[i_{k-1}, i_k]} \cdot \frac{1}{R[i_k, i_1]} < 1$, οπότε

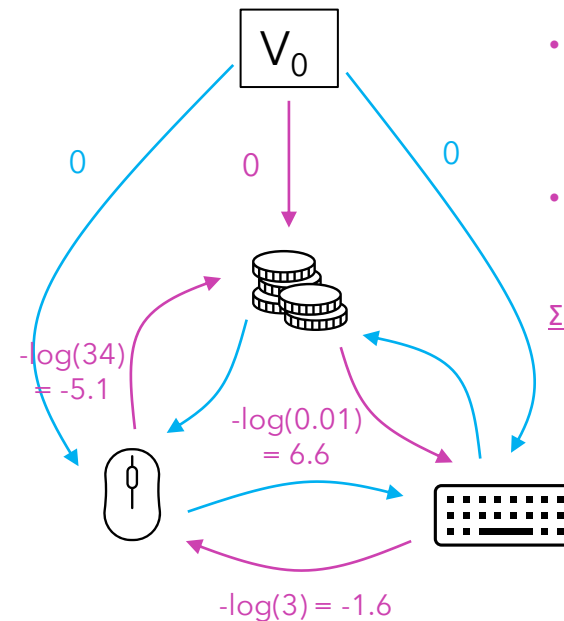
$\log\left(\frac{1}{R[i_1, i_2]} \cdot \frac{1}{R[i_2, i_3]} \dots \frac{1}{R[i_{k-1}, i_k]} \cdot \frac{1}{R[i_k, i_1]}\right) < \log(1)$

$\log\left(\frac{1}{R[i_1, i_2]}\right) + \log\left(\frac{1}{R[i_2, i_3]}\right) + \dots + \log\left(\frac{1}{R[i_{k-1}, i_k]}\right) + \log\left(\frac{1}{R[i_k, i_1]}\right) < 0$

Έτσι, το πρόβλημα, μετασχηματίζεται στην εύρεση κύκλου αρνητικού βάρους στον γράφο G . Για αυτό, μπορεί να χρησιμοποιηθεί ο αλγόριθμος Bellman-Ford και να επιστραφεί η συμπληρωματική απάντηση αυτής που δίνει, δηλαδή $\neg True = False$, αν δεν υπάρχει κανένας κύκλος αρνητικού βάρους ή $\neg False = True$, αν υπάρχει κάποιος κύκλος αρνητικού βάρους.

Επειδή οι ακμές που ξεκινούν από την κορυφή έναρξης έχουν μηδενικό βάρος, δεν προστίθενται νέοι κύκλοι αλλά αυτή συνδέεται με όλους τους κύκλους αρνητικού βάρους.

Απαιτείται χρόνος $\Theta(n^2)$ για τη δημιουργία του γράφου και $O(n^3)$ για την εκτέλεση του αλγορίθμου Bellman-Ford στη συνέχεια. Άρα, η διαδικασία συνολικά, έχει πολυπλοκότητα $O(n^3)$, που είναι πολυωνυμική, με μικρό σχετικά εκθέτη, και άρα ο αλγόριθμος είναι αποδοτικός.



- με 100 ευρώ αγοράζεται ένα πληκτρολόγιο, οπότε, με 1 μονάδα του ευρώ αγοράζονται $1/100 = 0.01$ μονάδες πληκτρολογίου => **$R[i_ευρώ, j_πληκτρολόγιο] = 0.01$**

- με 1 μονάδα πληκτρολογίου αγοράζονται 3 μονάδες ποντικών => **$R[i_πληκτρολόγιο, j_ποντίκι] = 3$**

- με 1 μονάδα ποντικού, 34 μονάδες ευρώ => **$R[i_ποντίκι, j_ευρώ] = 34$**

Συνολικά: $0.01 * 3 * 34 = \mathbf{1.02 > 1}$

Περιγράψτε έναν αποδοτικό αλγόριθμο που υπολογίζει τον μέγιστο αριθμό φοιτητών που μπορούν να εξεταστούν σε n ξεχωριστές αίθουσες χωρίς να χρησιμοποιούνται 3 συνεχόμενες αίθουσες. Η είσοδος του αλγορίθμου, είναι ένας πίνακας $S[1, \dots, n]$ όπου $S[i]$ είναι η χωρητικότητα της αίθουσας i .

- $\text{MaxS}(i,c) =$
 - 0, αν $i > n$ [είμαστε εκτός των ορίων του πίνακα]
 - $\text{MaxS}(i+1, 0)$, αν $c=2$ [δε χρησιμοποιείται η i-οστή αλλά οι 2 προηγούμενες της]
 - $\max\{$
 - $S[i] + \text{MaxS}(i+1, c+1)$, [χρησιμοποιείται η i-οστή]
 - $\text{MaxS}(i+1, 0)$ [δε χρησιμοποιείται η i-οστή]
 - $\}$
- Για κάθε αίθουσα i και δείκτη $c \in \{0,1,2\}$ που συμβολίζει τον αριθμό των συνεχόμενων αιθουσών που έχουν επιλεγθεί πριν την i .
- Η απάντηση είναι η τιμή $\text{MaxS}(1,0)$
- Χρησιμοποιούμε τις αίθουσες από των οποίων τη χωρητικότητα, $S[i]$, συνυπολογίζεται η μέγιστη τιμή που λαμβάνουμε κάθε φορά.
- Μπορούμε να αποθηκεύουμε τα αποτελέσματα σε έναν διδιάστατο πίνακα $\text{dp}[n][3]$.
- Πολυπλοκότητα χρόνου: $O(n)$.

Παράδειγμα με 8 αίθουσες (n=8):

✓	✓		✓	✓		✓	✓
3	2	5	10	7	8	4	6
1	2	3	4	5	6	7	8

dp =

32	0	0
29	29	0
29	27	27
27	24	21
21	17	14
14	14	10
10	6	6
6	6	0

$\text{MaxS}(1,0) = \max\{ \mathbf{S[1]} + \mathbf{MaxS(2,1)}, \text{MaxS}(2,0) \} = \max\{ 3+29, 29 \} = \mathbf{32}$
 $\text{MaxS}(2,0) = \max\{ S[2] + \text{MaxS}(3,1), \text{MaxS}(3,0) \} = \max\{ 2+27, 27 \} = 29$
 $\text{MaxS}(3,0) = \max\{ S[3] + \text{MaxS}(4,1), \text{MaxS}(4,0) \} = \max\{ 5+24, 27 \} = 29$
 $\mathbf{MaxS(4,0) = \max\{ \mathbf{S[4]} + \mathbf{MaxS(5,1)}, \text{MaxS}(5,0) \} = \max\{ 10+17, 21 \} = 27}$
 $\text{MaxS}(5,0) = \max\{ S[5] + \text{MaxS}(6,1), \text{MaxS}(6,0) \} = \max\{ 7+14, 12 \} = 21$
 $\text{MaxS}(6,0) = \max\{ S[6] + \text{MaxS}(7,1), \text{MaxS}(7,0) \} = \max\{ 8+6, 10 \} = 14$
 $\mathbf{MaxS(7,0) = \max\{ \mathbf{S[7]} + \mathbf{MaxS(8,1)}, \text{MaxS}(8,0) \} = \max\{ 4+6, 6 \} = 10}$
 $\text{MaxS}(8,0) = \max\{ S[8] + \text{MaxS}(9,1), \text{MaxS}(9,0) \} = \max\{ 6+0, 0 \} = 6$
 $\text{MaxS}(9,0) = 0$

$\text{MaxS}(8,2) = \text{MaxS}(9,0) = 0$
 $\text{MaxS}(7,2) = \text{MaxS}(8,0) = 6$
 $\mathbf{MaxS(6,2) = MaxS(7,0) = 10}$
 $\text{MaxS}(5,2) = \text{MaxS}(6,0) = 14$
 $\text{MaxS}(4,2) = \text{MaxS}(5,0) = 21$
 $\mathbf{MaxS(3,2) = MaxS(4,0) = 27}$

$\text{MaxS}(9,1) = 0$
 $\mathbf{MaxS(8,1) = \max\{ \mathbf{S[8]} + \mathbf{MaxS(9,2)}, \text{MaxS}(9,0) \} = \max\{ 6+0, 0 \} = 6}$
 $\text{MaxS}(7,1) = \max\{ S[7] + \text{MaxS}(8,2), \text{MaxS}(8,0) \} = \max\{ 4+0, 6 \} = 6$
 $\text{MaxS}(6,1) = \max\{ S[6] + \text{MaxS}(7,2), \text{MaxS}(7,0) \} = \max\{ 8+6, 10 \} = 14$
 $\mathbf{MaxS(5,1) = \max\{ \mathbf{S[5]} + \mathbf{MaxS(6,2)}, \text{MaxS}(6,0) \} = \max\{ 7+10, 14 \} = 17}$
 $\text{MaxS}(4,1) = \max\{ S[4] + \text{MaxS}(5,2), \text{MaxS}(5,0) \} = \max\{ 10+14, 21 \} = 24$
 $\text{MaxS}(3,1) = \max\{ S[3] + \text{MaxS}(4,2), \text{MaxS}(4,0) \} = \max\{ 5+21, 27 \} = 27$
 $\mathbf{MaxS(2,1) = \max\{ \mathbf{S[2]} + \mathbf{MaxS(3,2)}, \text{MaxS}(3,0) \} = \max\{ 2+27, 27 \} = 29}$

Ως υπεύθυνος δικτύου μιας αλυσίδας εστιατορίων, πρέπει να επιλέξετε που θα ανοίξουν καταστήματα κατά μήκος της νέας εθνικής οδού. Έχουν προεπιλεγεί n υποψήφιας θέσεις, και για κάθε υποψήφια θέση i , $i = 1, \dots, n$, έχει υπολογισθεί το αναμενόμενο κέρδος από το κατάστημα στη θέση i με βάση το αν ανοίξουν καταστήματα στις γειτονικές θέσεις. Αν δεν ανοίξει κατάστημα σε καμία από τις θέσεις $i - 1$ και $i + 1$, το αναμενόμενο κέρδος από το κατάστημα στην θέση i είναι a_i , αν ανοίξει κατάστημα σε μία από τις θέσεις $i - 1$ και $i + 1$, το αναμενόμενο κέρδος είναι b_i , και αν ανοίξουν καταστήματα σε αμφότερες τις θέσεις $i - 1$ και $i + 1$, το αναμενόμενο κέρδος είναι c_i (τα c_1 και c_n δεν ορίζονται, και για κάθε θέση i , ισχύει ότι $a_i \geq b_i \geq c_i \geq 0$). Να διατυπώσετε έναν όσο το δυνατόν πιο αποδοτικό αλγόριθμο, που με είσοδο τις τριάδες (a_i, b_i, c_i) , $i = 1, \dots, n$, επιλέγει τις θέσεις όπου θα ανοίξουν καταστήματα ώστε να μεγιστοποιηθεί το κέρδος.

Λύση.

Για κάθε $i = 1, \dots, n$, θεωρούμε το υποσύνολο $A_i = \{1, \dots, i\}$ που αποτελείται από τις πρώτες i θέσεις. Θα διατυπώσουμε μια αναδρομική σχέση που συνδέει το βέλτιστο κέρδος από τις θέσεις στο σύνολο A_i με το βέλτιστο κέρδος από τις θέσεις στο σύνολο A_{i-1} . Η διαφορά σε σχέση με τα προηγούμενα προβλήματα είναι ότι το επιπλέον κέρδος από το κατάστημα στη θέση i εξαρτάται από το αν ανοίγουν καταστήματα στις θέσεις $i-1$ και $i+1$. Για αυτό διακρίνουμε τρεις διαφορετικές περιπτώσεις σε σχέση με το άνοιγμα καταστημάτων στις θέσεις i και $i+1$, και υπολογίζουμε διαφορετικά το βέλτιστο κέρδος για καθεμία από αυτές. Συγκεκριμένα, έστω ότι:

- $P_0(i)$ είναι το βέλτιστο κέρδος από τις θέσεις στο σύνολο A_i δεδομένου ότι δεν ανοίγει κατάστημα στη θέση i .
- $P_{1,0}(i)$ είναι το βέλτιστο κέρδος από τις θέσεις στο σύνολο A_i δεδομένου ότι ανοίγει κατάστημα στη θέση i και δεν ανοίγει κατάστημα στη θέση $i+1$.
- $P_{1,1}(i)$ είναι το βέλτιστο κέρδος από τις θέσεις στο σύνολο A_i δεδομένου ότι ανοίγει κατάστημα τόσο στη θέση i όσο και στη θέση $i+1$.

Για $i = 1$, προφανώς έχουμε ότι:

$$P_0(1) = 0, \quad P_{1,0}(1) = a_1, \quad P_{1,1}(1) = b_1$$

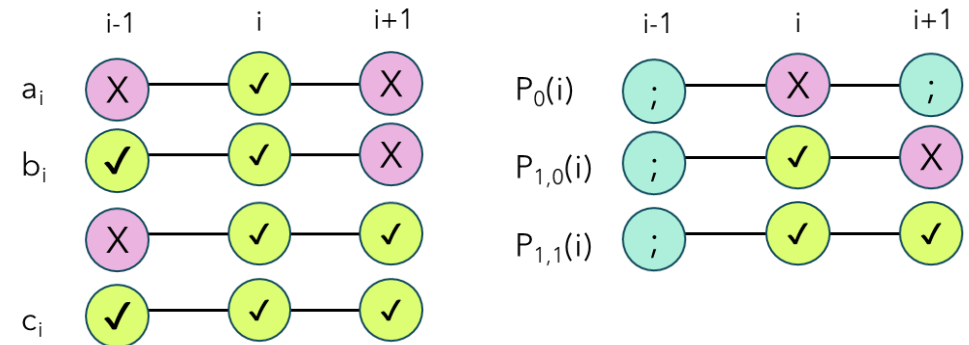
Για $i \geq 2$, τα $P_0(i)$, $P_{1,0}(i)$, και $P_{1,1}(i)$ υπολογίζονται αναδρομικά από τα $P_0(i-1)$, $P_{1,0}(i-1)$, και $P_{1,1}(i-1)$ ως εξής:

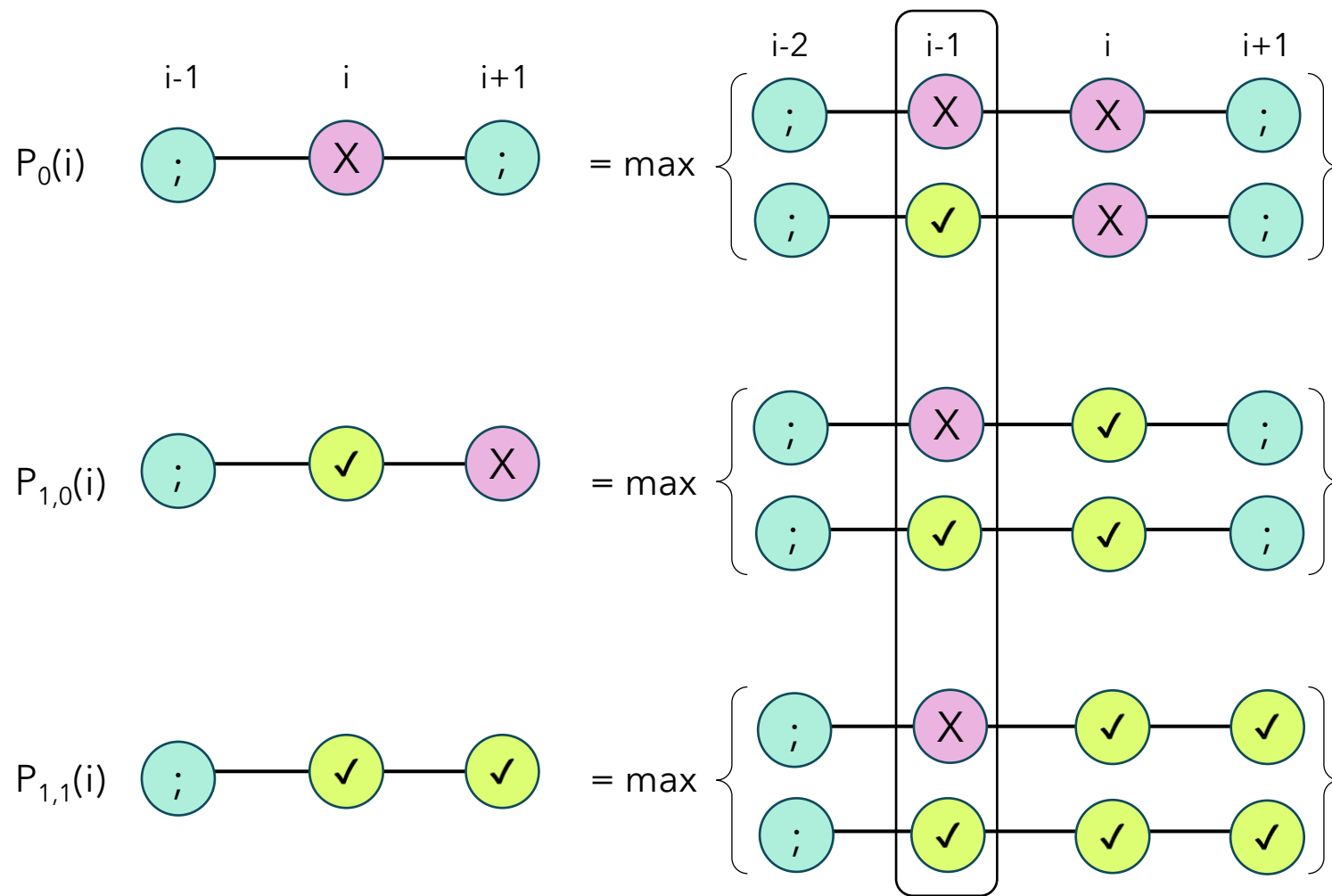
$$P_0(i) = \max\{P_0(i-1), P_{1,0}(i-1)\}$$

$$P_{1,0}(i) = \max\{a_i + P_0(i-1), b_i + P_{1,1}(i-1)\}$$

$$P_{1,1}(i) = \max\{b_i + P_0(i-1), c_i + P_{1,1}(i-1)\}$$

Το μέγιστο κέρδος είναι $\max\{P_0(n), P_{1,0}(n)\}$. Το μέγιστο κέρδος και η βέλτιστη λύση μπορούν να υπολογιστούν με δυναμικό προγραμματισμό σε χρόνο $\Theta(n)$. $\square$





Θέμα 6 [30 μονάδες]

Το νοσοκομείο μιας πόλης απέκτησε πρόσφατα έναν υπερσύγχρονο μαγνητικό τομογράφο, τον οποίο διαθέτει προς χρήση στις κλινικές του νοσοκομείου για την καλύτερη διάγνωση των παθήσεων των ασθενών τους. Το τμήμα χειρισμού του νέου μαγνητικού τομογράφου λαμβάνει ως εκ τούτου ένα πολύ μεγάλο πλήθος n αιτημάτων διαγνωστικής εξέτασης ασθενών ($\{1, 2, \dots, n\}$), τα οποία όμως δεν είναι δυνατόν να εξυπηρετήσει στη διάρκεια μιας ημέρας. Για τον λόγο αυτό, αποφάσισε να εξυπηρετεί (στη διάρκεια μιας ημέρας) τα αιτήματα εκείνα τα οποία έχουν την πιο επείγουσα διαγνωστική ανάγκη: για κάθε αίτημα i που λαμβάνει με χρόνο έναρξης της εξέτασης τη χρονική στιγμή ϵ_i και χρόνο λήξης της εξέτασης τη χρονική στιγμή λ_i , συσχετίζει μια τιμή βαρύτητας β_i στην κλίμακα 1 (μη επείγουσα) έως 10 (κατεπείγουσα) που αποτιμά τον βαθμό επείγοντος της αιτούμενης εξέτασης. Προφανώς, δύο αιτήματα i και j με επικαλυπτόμενα χρονικά διαστήματα ($[\epsilon_i, \lambda_i) \cap [\epsilon_j, \lambda_j) \neq \emptyset$) δεν μπορούν να εξυπηρετηθούν και τα δύο. Τέτοια αιτήματα ονομάζονται ασύμβατα. Το πρόβλημα του τμήματος χειρισμού του νέου μαγνητικού τομογράφου είναι να ικανοποιήσει ένα υποσύνολο $\Sigma \subseteq \{1, 2, \dots, n\}$ ανά δύο χρονικά συμβατών αιτημάτων, το οποίο να μεγιστοποιεί το άθροισμα της επείγουσας βαρύτητας των διαγνωστικών αιτημάτων του Σ .

Για παράδειγμα, έστω ότι το τμήμα του νέου μαγνητικού τομογράφου ανοίγει στις 9:00 π.μ. και δέχεται αιτήματα διαγνωστικής χρήσης με χρόνους έναρξης και λήξης που μετρούνται σε λεπτά μετά τις 9:00 π.μ.. Δηλαδή, το αίτημα χρήσης [95, 120) αφορά στη χρήση του τομογράφου από τις 10:35 π.μ. έως τις 10:59 π.μ.. Θεωρήστε λοιπόν ότι έχουν $n = 8$ αιτήματα, με χρονικά διαστήματα έναρξης και λήξης και βαρύτητα (βαθμό επείγοντος) διαγνωστικής εξέτασης:

1. Θεωρήστε τον εξής άπληστο αλγόριθμο επίλυσης του παραπάνω προβλήματος: ταξινομεί τα αιτήματα κατά φθίνουσα σειρά επείγουσας βαρύτητας, επιλέγει το πρώτο, μετά επιλέγει το επόμενο συμβατό αίτημα στη φθίνουσα ακολουθία, κατόπιν το μεθεπόμενο συμβατό αίτημα, κ.ο.κ.. Είναι ο άπληστος αυτός αλγόριθμος βέλτιστος; Αν ναι, αποδείξτε ότι βρίσκει πάντοτε τη βέλτιστη λύση. Αν όχι, δώστε ένα αντιπαράδειγμα για το οποίο ο αλγόριθμος βρίσκει μια μη βέλτιστη λύση.
2. Βρείτε την τιμή (συνολικό άθροισμα των βαρυτήτων) της βέλτιστης λύσης του παραπάνω προβλήματος, αναπτύσσοντας έναν αλγόριθμο δυναμικού προγραμματισμού. Η περιγραφή του αλγορίθμου μπορεί να είναι σε άτυπη μορφή (φυσική γλώσσα), αλλά πρέπει να περιλαμβάνει οπωσδήποτε την/τις αναδρομική/ές σχέση/εις που διέπουν τον αλγόριθμο και συμπληρώνουν τον πίνακα δυναμικού προγραμματισμού.
3. (α') Δώστε τον χρόνο εκτέλεσης του αλγορίθμου σας, ο οποίος πρέπει να είναι πολυωνυμικός ως προς το πλήθος n των αιτημάτων.
(β') Εκτελέστε τον αλγόριθμό σας στο παραπάνω παράδειγμα δίνοντας τις τιμές του πίνακα δυναμικού προγραμματισμού σε κάθε βήμα.

Για παράδειγμα, έστω ότι το τμήμα του νέου μαγνητικού τομογράφου ανοίγει στις 9:00 π.μ. και δέχεται αιτήματα διαγνωστικής χρήσης με χρόνους έναρξης και λήξης που μετρούνται σε λεπτά μετά τις 9:00 π.μ.. Δηλαδή, το αίτημα χρήσης $[95, 120)$ αφορά στη χρήση του τομογράφου από τις 10:35 π.μ. έως τις 10:59 π.μ.. Θεωρήστε λοιπόν ότι έχουν $n = 8$ αιτήματα, με χρονικά διαστήματα έναρξης και λήξης και βαρύτητα (βαθμό επείγοντος) διαγνωστικής εξέτασης:

- $[\epsilon_1, \lambda_1) = [0, 20), \beta_1 = 3$
- $[\epsilon_2, \lambda_2) = [10, 25), \beta_2 = 7$
- $[\epsilon_3, \lambda_3) = [25, 50), \beta_3 = 7$
- $[\epsilon_4, \lambda_4) = [15, 60), \beta_4 = 8$
- $[\epsilon_5, \lambda_5) = [40, 70), \beta_5 = 5$
- $[\epsilon_6, \lambda_6) = [50, 70), \beta_6 = 5$
- $[\epsilon_7, \lambda_7) = [70, 80), \beta_7 = 9$
- $[\epsilon_8, \lambda_8) = [75, 90), \beta_8 = 9$

Τότε, (ενδεικτικές) εφικτές λύσεις είναι τα υποσύνολα αιτημάτων $\{1, 3, 6, 7\}$, $\{2, 8\}$ και $\{2, 3, 6, 7\}$, με συνολικό βαθμό επείγουσας βαρύτητας 24, 16, και 28 αντίστοιχα.

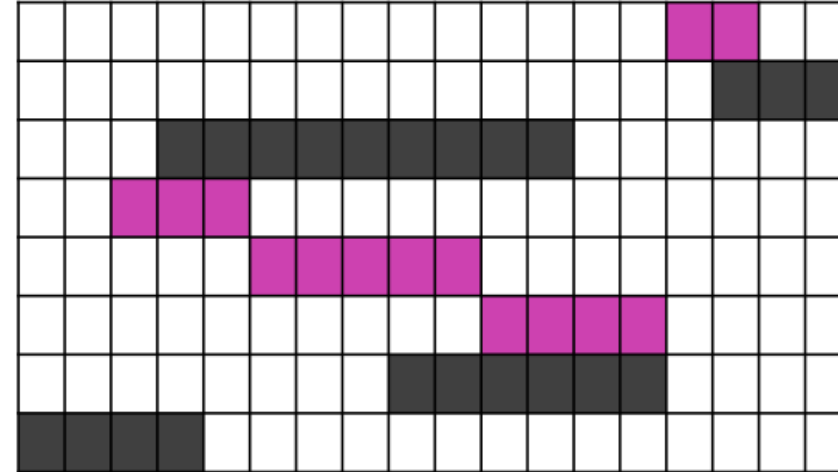
1. Ο άπληστος αλγόριθμος δεν είναι βέλτιστος. Στο παράδειγμα της εκφώνησης, ανακαλύπτει είτε την εφικτή λύση $\{2, 5, 7\}$ με συνολική αθροιστική βαρύτητα 21, είτε την εφικτή λύση $\{4, 8\}$ με συνολική αθροιστική βαρύτητα 17, ενώ η εφικτή λύση $\{2, 3, 6, 7\}$ έχει συνολική αθροιστική βαρύτητα $28 > \max\{21, 17\}$.
2. Διατάσσουμε τα αιτήματα κατά αύξοντα χρόνο λήξης (αν δύο ή περισσότερα αιτήματα έχουν ταυτόσημο χρόνο λήξης, τότε η μεταξύ τους διάταξη είναι αυθαίρετη). Έστω ότι το $p(i)$ συμβολίζει τη μέγιστη τιμή $j < i$, για την οποία το αίτημα i είναι συμβατό με το αίτημα j (αν δεν υπάρχει j με αυτή την ιδιότητα, τότε $p(i) = 0$). Καταρχάς, παρατηρούμε ότι αν η βέλτιστη λύση περιέχει το n -οστό αίτημα με βαρύτητα β_n , τότε τα (ασύμβατα) αιτήματα $\{p(n) + 1, p(n) + 2, \dots, n - 1\}$ δεν μπορούν να συμπεριληφθούν στη βέλτιστη λύση. Διαφορετικά, αν η βέλτιστη λύση δεν περιέχει το n -οστό αίτημα, τότε είναι ίδια με τη βέλτιστη λύση των αιτημάτων $\{1, 2, \dots, n - 1\}$. Έστω ότι το $X[i]$ συμβολίζει την τιμή (συνολικό άθροισμα βαρυτήτων) της βέλτιστης λύσης των αιτημάτων $\{1, 2, \dots, i\}$. Τότε, με βάση τον παραπάνω συλλογισμό, προκύπτει η αναδρομική σχέση $X[i] = \max\{\beta_i + X[p(i)], X[i - 1]\}$ με $X[0] = 0$. Το συνολικό άθροισμα των βαρυτήτων της βέλτιστης λύσης είναι η τιμή του $X[n]$.

Αύξουσα ταξιν. βάσει
του χρόνου λήξης:

- $[\epsilon_1, \lambda_1) = [0, 20), \beta_1 = 3$
- $[\epsilon_2, \lambda_2) = [10, 25), \beta_2 = 7$
- $[\epsilon_3, \lambda_3) = [25, 50), \beta_3 = 7$
- $[\epsilon_4, \lambda_4) = [15, 60), \beta_4 = 8$
- $[\epsilon_5, \lambda_5) = [40, 70), \beta_5 = 5$
- $[\epsilon_6, \lambda_6) = [50, 70), \beta_6 = 5$
- $[\epsilon_7, \lambda_7) = [70, 80), \beta_7 = 9$
- $[\epsilon_8, \lambda_8) = [75, 90), \beta_8 = 9$

Φθίνουσα ταξιν. βάσει
της βαρύτητας:

- $[\epsilon_7, \lambda_7) = [70, 80), \beta_7 = 9$
- $[\epsilon_8, \lambda_8) = [75, 90), \beta_8 = 9$
- $[\epsilon_4, \lambda_4) = [15, 60), \beta_4 = 8$
- $[\epsilon_2, \lambda_2) = [10, 25), \beta_2 = 7$
- $[\epsilon_3, \lambda_3) = [25, 50), \beta_3 = 7$
- $[\epsilon_6, \lambda_6) = [50, 70), \beta_6 = 5$
- $[\epsilon_5, \lambda_5) = [40, 70), \beta_5 = 5$
- $[\epsilon_1, \lambda_1) = [0, 20), \beta_1 = 3$



28

3. (α') Για τον χρόνο εκτέλεσης του αλγορίθμου, παρατηρήστε ότι πρέπει να υπολογίσουμε n τιμές του πίνακα X , όπου καθεμία απαιτεί ένα σταθερό πλήθος πράξεων (μια πρόσθεση και μια εύρεση μεγίστου). Άρα, η συνολική χρονική πολυπλοκότητα υπολογισμού των τιμών του X είναι $O(n)$. Επίσης, χρειαζόμαστε ταξινόμηση των αιτημάτων κατά αύξοντα χρόνο λήξης, από την οποία μπορούμε με δυαδική αναζήτηση να υπολογίσουμε για κάθε αίτημα i την τιμή $p(i)$ (μέγιστη τιμή $j < i$, για την οποία το αίτημα i είναι συμβατό με το αίτημα j). Άρα, ο συνολικός χρόνος εκτέλεσης του αλγορίθμου είναι $O(n \log n)$.
- (β') Με βάση την παραπάνω ταξινόμηση των αιτημάτων κατά αύξοντα χρόνο λήξης (το οποίο ισχύει για τα δοθέντα αιτήματα του παραδείγματος), παρατηρούμε ότι $p(1) = 0, p(2) = 0, p(3) = 2, p(4) = 0, p(5) = 2, p(6) = 3, p(7) = 6, p(8) = 6$. Η εκτέλεση του αλγορίθμου στο παράδειγμα έχει ως εξής:
- $X[0] = 0$
 - $X[1] = \max\{3 + X[p(1)], X[0]\} = \max\{3 + 0, 0\} = 3$
 - $X[2] = \max\{7 + X[p(2)], X[1]\} = \max\{7 + 0, 3\} = 7$
 - $X[3] = \max\{7 + X[p(3)], X[2]\} = \max\{7 + 7, 7\} = 14$
 - $X[4] = \max\{8 + X[p(4)], X[3]\} = \max\{8 + 0, 14\} = 14$
 - $X[5] = \max\{5 + X[p(5)], X[4]\} = \max\{5 + 7, 14\} = 14$
 - $X[6] = \max\{5 + X[p(6)], X[5]\} = \max\{5 + 14, 14\} = 19$
 - $X[7] = \max\{9 + X[p(7)], X[6]\} = \max\{9 + 19, 19\} = 28$
 - $X[8] = \max\{9 + X[p(8)], X[7]\} = \max\{9 + 19, 28\} = 28$

Θέμα 5 [35 μονάδες]

Δίνεται $a_1, a_2, \dots, a_n$ πεπερασμένη ακολουθία αριθμών μήκους $n > 1$. Η ακολουθία (a_i) θα λέμε ότι έχει γνήσια αύξουσα υπακολουθία αν υπάρχουν $k, l \in \{1, \dots, n\}$, με $k \leq l$, τέτοια ώστε $a_k < a_{k+1} < \dots < a_l$. Να δώσετε αλγόριθμο δυναμικού προγραμματισμού που να υπολογίζει το μήκος της μεγαλύτερης γνήσια αύξουσας υπακολουθίας αριθμών καθώς και τη θέση του πρώτου στοιχείου της.

a_1	a_2	...	...	...	a_{n-1}	a_n
-------	-------	-----	-----	-----	-----------	-------



Ακολουθία:

α_1	α_2	...	...	...	α_{n-1}	α_n
------------	------------	-----	-----	-----	----------------	------------

Μαχ. μήκος:

m_1	m_2	...	...	...	m_{n-1}	m_n
-------	-------	-----	-----	-----	-----------	-------

Τελευταία θέση:

l_1	l_2	...	...	...	l_{n-1}	l_n
-------	-------	-----	-----	-----	-----------	-------

Θέση έναρξης;

- αν δε μας ενδιαφέρει η πληροφορία του μέγιστου μήκους και της τελευταίας θέσης για κάθε υπο-πρόβλημα, τότε οι αντίστοιχοι πίνακες θα μπορούσαν να έχουν μόνο μία θέση.

Δημιουργείται ένας πίνακας c , μήκους n , που αρχικοποιείται με τιμή 1 σε κάθε θέση (ελάχιστο μήκος = 1 αριθμός), και δύο μεταβλητές, max_len , max_end (θα μπορούσαν να είναι δύο πίνακες μήκους n , αν μας ενδιέφερε η αποθήκευση των αντίστοιχων τιμών σε κάθε βήμα της διαδικασίας), που αρχικοποιούνται με τιμή 1, όπου αποθηκεύονται αντίστοιχα, το τρέχον μέγιστο μήκος γνησίως αύξουσας υπακολουθίας και ο αύξων αριθμός της θέσης όπου αυτή έχει το πέρας της. Στη συνέχεια, υπολογίζεται η τιμή για κάθε θέση, ξεκινώντας από τη δεύτερη, αφού της πρώτης δεν προηγείται κανένας αριθμός. Σε κάθε θέση i , ελέγχεται αν $a_{i-1} < a_i$, και στην περίπτωση που ισχύει, θέτουμε $c[i] = c[i-1] + 1$ και ελέγχουμε αν είναι μεγαλύτερη της max_len , περίπτωση κατά την οποία αποθηκεύεται σε αυτήν η νέα τιμή, $c[i]$, και τίθεται $\text{max_end} = i$. Διαφορετικά, προχωρούμε στην επόμενη θέση, αν δεν έχουμε φτάσει στην τελευταία. Με την ολοκλήρωση αυτής της διαδικασίας, στη μεταβλητή max_len έχουμε το μήκος της μέγιστης γνησίως αύξουσας υπακολουθίας και η θέση έναρξής της είναι η $(\text{max_end} - \text{max_len} + 1)$.

Στη διαδικασία που περιγράφηκε, εκτελούνται ακριβώς n βήματα για τη δημιουργία και αρχικοποίηση του πίνακα, από 1 για τη δημιουργία και αρχικοποίηση των μεταβλητών και $(n - 1)$ για τους ελέγχους στις θέσεις 2 έως και n του πίνακα c και πάντοτε τερματίζει καθώς γίνεται μόνο μία σύγκριση με την προηγούμενη θέση και με τη μεταβλητή max_len . Μάλιστα, με τον τερματισμό, οπότε θα έχει ελεγχθεί και το n -οστό στοιχείο της ακολουθίας, θα οδηγήσει στη σωστή απάντηση, μιας και πρέπει να είναι συνεχόμενα τα στοιχεία της υπακολουθίας / δεν μπορούμε να έχουμε κενό μεταξύ τους, άρα κάθε φορά η δεδομένη υπακολουθία αποτυπώνει το μέγιστο μήκος της και κρατάμε ως τελική λύση τη μέγιστη τιμή από αυτές.

- 6.** Σχεδιάστε έναν αλγόριθμο Δυναμικού Προγραμματισμού για την εύρεση της μέγιστης μονότονα αύξουσας υποακολουθίας μιας ακολουθίας n αριθμών σε χρόνο $O(n^2)$.

Λύση

Ορίζουμε ως $L(i)$ το μήκος της μέγιστης αύξουσας υποακολουθίας η οποία έχει σαν τελευταίο στοιχείο το $a(i)$. Η αρχική μας συνθήκη είναι

$$L(i) = 1, \quad \forall i$$

Η αναδρομική μας εξίσωση είναι η

$$L(j) = \max_{\substack{i < j \\ a(i) < a(j)}} \{L(i)\} + 1, \quad \text{if } L(j) \leq L(i)$$

Πιο αναλυτικά, αρχικά θεωρούμε ότι κάθε στοιχείο της δοσμένης ακολουθίας αποτελεί από μόνο του μια αύξουσα υποακολουθία μήκους 1, εξού και η αρχική συνθήκη $L(i) = 1, \forall i$. Στη

Αλγόριθμος 6 Longest Increasing Subsequence

```
1: INPUT: Array  $a$  of integers
2:
3:  $n \leftarrow \text{length}(a)$ 
4:  $L \leftarrow$  Vector (size  $n$ ) of increasing subsequences
5: Insert  $a[0]$  to  $L[0]$ 
6:
7: for  $j \leftarrow 1..n - 1$  do
8:   for  $i \leftarrow 0..j - 1$  do
9:     if  $a[i] < a[j]$  and  $\text{length}(L[j]) \leq \text{length}(L[i])$  then
10:        $L[j] \leftarrow L[i]$ 
11:     end if
12:   end for
13:
14:    $L[j] \leftarrow L[j] \cup a[j]$ 
15: end for
16:
17:  $L_{max} \leftarrow L[k] : \text{length}(L[k]) \geq \text{length}(L[i]), \quad \forall i \in \{0..n - 1\}$ 
18: return  $L_{max}$ 
```

συνέχεια, ξεκινώντας από το δεύτερο στοιχείο της αρχικής ακολουθίας ($j = 1$) και πηγαίνοντας δεξιότερα, ελέγχουμε όλα τα στοιχεία που βρίσκονται στα αριστερά ($i < j$) και τα οποία έχουν μικρότερη τιμή από το τρέχον στοιχείο ($a(i) < a(j)$). Εφόσον βρεθεί κάποιο τέτοιο i , κοιτάμε το $L(i)$ και στην περίπτωση που έχουμε ότι $L(j) \leq L(i)$, τότε το στοιχείο $a(j)$ μπορεί να ενσωματωθεί σε αυτήν την υποακολουθία, αυξάνοντας το μήκος της κατά 1 ($L(j) = L(i) + 1$).

Παρατήρηση: Η τελευταία πράξη έχει νόημα, μόνο αν $L(j) \leq L(i)$, αλλιώς η $L(j)$ είναι ούτως ή άλλως μεγαλύτερη υποακολουθία και δεν υπάρχει λόγος να γίνει καμμία ενέργεια.

Τελικά, αφού έχουμε σαρώσει όλα τα στοιχεία της αρχικής ακολουθίας, εντοπίζουμε το στοιχείο k για το οποίο το $L(k)$ είναι μέγιστο και επιστρέφουμε την τιμή του $L(k)$.

Με το παραπάνω σκεπτικό απλώς βρήκαμε το μήκος της μέγιστης αύξουσας υποακολουθίας. Αν θέλουμε να βρούμε και να επιστρέψουμε ακριβώς τα στοιχεία που αποτελούν τη μέγιστη αύξουσα υποακολουθία, θα ακολουθήσουμε το ίδιο εν γένει σκεπτικό όπως και παραπάνω, αλλά με κάποιες μικρές τροποποιήσεις. Για παράδειγμα, πλέον κάθε $L(i)$ θα συμβολίζει όχι το μήκος της μέγιστης αύξουσας υποακολουθίας που τελειώνει στο i , αλλά το σύνολο όλων των στοιχείων της υποακολουθίας αυτής. Συνεπώς και η αναδρομική μας σχέση τροποποιείται ως εξής:

Η αρχική μας συνθήκη γίνεται

$$L(i) = \{a(i)\}, \quad \forall i$$

και

$$L(j) = \left(\max_{\substack{i < j \\ a(i) < a(j)}} L(i) \right) \cup a(j), \quad \text{if } |L(j)| \leq |L(i)|$$

όπου σε αυτή την περίπτωση, η συνάρτηση $\max$ έχει τροποποιηθεί ελαφρώς για να επιστρέφει το σύνολο με τα περισσότερα στοιχεία.

Ο αλγόριθμος 6 αποτελεί υλοποίηση αυτής της ιδέας και η πολυπλοκότητά του είναι $O(n^2)$.

ΣΥΝΕΧΕΙΑ ΣΤΟ ΕΠΟΜΕΝΟ ΦΡΟΝΤΙΣΤΗΡΙΟ...



Πηγές Φ10:

Σημειώσεις / διαλέξεις μαθήματος, προτεινόμενα βιβλία, υλικό μαθήματος προηγούμενων ετών,
SMA5503, Michael Sambol, Δ. Φωτάκης